

Backend Documentation

@Douglas Silva @Fernando Bach

Cronograma do Backend (API Flask)

Equipe: Douglas Clayton da Silva & Fernando Bach

Objetivo: Desenvolver uma API REST em Flask (Python) que recebe métricas e logs do agente, persiste em PostgreSQL, aplica regras de detecção de eventos de segurança e fornece dados para o dashboard.

July

17

Estrutura do Cronograma

Semana	Período	Foco Principal
1	21/04 – 27/04	Estrutura do projeto e endpoints mock
2	28/04 – 04/05	Integração com banco de dados
3	05/05 – 11/05	Endpoints de consulta e persistência de logs
4	12/05 – 18/05	Processamento de logs e extração de campos
5	19/05 – 25/05	Regras de detecção (Brute Force) e alertas
6	26/05 – 01/06	Regra de Port Scan + Preparação parcial
7	02/06 – 05/06	Estabilidade, documentação e ajustes finais

♦ Semana 1 | 21 – 27 abr

 **Meta da Semana:** Ter o projeto Flask rodando em Docker com endpoints mock definidos e contrato JSON acordado com as demais equipes.

Tarefas

- **1.1** Criar repositório Git (ou pasta dentro do repositório principal) para o backend.
- **1.2** Inicializar projeto Flask com `requirements.txt` incluindo:
 - `Flask`
 - `Flask-SQLAlchemy` (ou `psycopg2-binary`)
 - `python-dotenv`
- **1.3** Criar `Dockerfile` para o backend:
 - Base `python:3.11-slim`
 - Copiar código, instalar dependências, expor porta 5000.
- **1.4** Adicionar serviço `backend` ao `docker-compose.yml` (junto com o banco).
- **1.5** Criar endpoint `GET /health` que retorna `{"status": "ok"}`.
- **1.6 Definir com o Agente** o formato exato do payload de métricas Ex.:

```
1 {
2   "host_id": "client-01",
3   "timestamp": "2026-04-21T14:35:00Z",
4   "cpu_percent": 23.5,
5   "memory_percent": 45.2,
6   "disk_percent": 60.0,
7   "net_sent": 1024,
8   "net_recv": 2048
9 }
10
```

- **1.7** Criar endpoint `POST /api/metrics` que apenas loga o JSON recebido e retorna `201 Created`.
- **1.8** Definir com o Agente e Banco o formato do payload de logs (a ser usado a partir da Semana 3).

Entregáveis da Semana

- API Flask rodando em container Docker.
- Endpoint `POST /api/metrics` funcional (mock).
- Contrato JSON de métricas documentado e compartilhado.

♦ Semana 2 | 28 abr – 04 mai

 **Meta da Semana:** Conectar a API ao PostgreSQL e salvar as métricas recebidas.

Tarefas

- **2.1** Configurar a conexão com PostgreSQL usando SQLAlchemy.
 - Ler `DATABASE_URL` de variável de ambiente.

- **2.2** Definir modelo `Host` (`id`, `hostname`, `ip_address`, `created_at`).
- **2.3** Definir modelo `Metric` (`id`, `host_id`, `timestamp`, `cpu_percent`, `memory_percent`, `disk_percent`, `net_sent`, `net_recv`).
- **2.4** No endpoint `POST /api/metrics` :
 - Validar campos obrigatórios (`host_id`, `timestamp`).
 - Buscar ou criar o `Host` correspondente.
 - Salvar a métrica no banco.
 - Retornar `201 Created` com ID da métrica.
- **2.5** Criar endpoint `GET /api/metrics` com parâmetros:
 - `host_id` (obrigatório)
 - `limit` (padrão 20)
 - `offset` (padrão 0)
 - Retornar lista de métricas ordenadas por `timestamp` decrescente.
- **2.6** Testar integração real:
 - Subir backend e banco via `docker compose up`.
 - Enviar requisição POST com dados de exemplo (via Postman ou script Python).
 - Verificar se os dados aparecem no banco.

✓ Entregáveis da Semana

- Métricas sendo persistidas no PostgreSQL.
- Endpoint de consulta de métricas funcionando.
- Integração Backend ↔ Banco validada.

♦ Semana 3 | 05 – 11 mai

🎯 **Meta da Semana:** Criar endpoints para recebimento e consulta de logs.

📋 Tarefas

- **3.1** Definir com o Agente o payload de logs. Exemplo:

```

1 {
2   "host_id": "client-01",
3   "timestamp": "2026-04-21T14:35:00Z",
4   "log_type": "auth",
5   "raw_line": "Apr 21 14:35:00 server sshd[123]: Failed password for root from 192.168.1.100"
6 }
7
```

- **3.2** Criar modelo `LogEntry` :
 - `id`, `host_id`, `timestamp`, `log_type`, `raw_line`, `parsed_data` (JSONB).

- **3.3** Criar endpoint `POST /api/logs` :
 - Receber payload e salvar na tabela `logs` .
 - Retornar `201 Created` .
- **3.4** Criar endpoint `GET /api/logs` :
 - Parâmetros: `host_id` (obrigatório), `limit` , `offset` , `log_type` (opcional).
 - Retornar logs ordenados por timestamp decrescente.
- **3.5** Testar envio de logs reais do agente (se disponível) ou simular com script.

✓ Entregáveis da Semana

- Logs sendo recebidos e persistidos.
 - Endpoint de consulta de logs funcional.
-

♦ Semana 4 | 12 – 18 mai

🎯 **Meta da Semana:** Processar logs de autenticação e extrair campos relevantes (IP, usuário, status).

📋 Tarefas

- **4.1** Criar função `parse_ssh_log(raw_line: str) -> dict` que extrai:
 - `ip_origem` (endereço IP)
 - `usuario` (nome do usuário)
 - `status` ("failed" ou "accepted")
 - `timestamp` (extraído da linha ou usar timestamp do agente)
- **4.2** No endpoint `POST /api/logs` , chamar a função de parsing e armazenar o resultado no campo `parsed_data` (JSONB).
- **4.3** Garantir que logs que não casam com o padrão SSH sejam salvos com `parsed_data = NULL` .
- **4.4** Criar uma consulta SQL (via SQLAlchemy) que conta tentativas de login falhas por IP nas últimas X horas (para uso futuro na detecção).
- **4.5** Testar com exemplos reais de `/var/log/auth.log` .

✓ Entregáveis da Semana

- Parsing de logs SSH funcional.
 - Dados estruturados disponíveis no campo `parsed_data` .
-

♦ Semana 5 | 19 – 25 mai 🔥 CRÍTICO

🎯 **Meta da Semana:** Implementar regra de detecção de Brute Force e sistema de alertas.

📋 Tarefas

- **5.1** Criar modelo `Alert` :
 - `id`, `host_id`, `alert_type` (`brute_force`, `port_scan`), `source_ip`, `timestamp`, `severity`, `resolved` (boolean).
- **5.2** Implementar lógica de detecção de Brute Force:
 - A cada novo log de falha SSH, contar quantas falhas do mesmo IP ocorreram nos últimos **60 segundos**.
 - Se $\geq$ **5 falhas**, criar um novo alerta (se não existir um ativo para o mesmo IP).
- **5.3** Criar endpoint `GET /api/alerts` :
 - Parâmetros: `status` (`active`, `resolved`, `all`), `host_id`.
 - Retornar lista de alertas ordenados por timestamp decrescente.
- **5.4** Criar endpoint `PATCH /api/alerts/<id>/resolve` para marcar alerta como resolvido.
- **5.5** Testar detecção com script de simulação do Agente (Hydra ou script bash).

✅ Entregáveis da Semana

- Regra de Brute Force funcionando.
- Alertas sendo gerados e consultáveis via API.

♦ Semana 6 | 26 mai – 01 jun ⚠️ Apresentação Parcial 29/05

🎯 **Meta da Semana:** Implementar regra de Port Scan e garantir que a API está estável para a apresentação parcial.

📋 Tarefas

- **6.1** Definir com o Agente o payload para eventos de rede (conexões ativas). Exemplo:

```
1 {
2   "host_id": "client-01",
3   "timestamp": "...",
4   "connections": [
5     {"local_port": 22, "remote_ip": "192.168.1.100", "state": "SYN_RECV"}
6   ]
7 }
8
```

- **6.2** Criar endpoint `POST /api/network` para receber esses dados.
- **6.3** Implementar detecção de Port Scan:

- Para cada IP remoto, contar quantas portas distintas foram acessadas nos últimos **30 segundos**.
- Se ≥ 10 **portas distintas**, gerar alerta do tipo `port_scan`.
- **6.4** Refatorar código usando **Blueprints** do Flask:
 - `metrics_bp`, `logs_bp`, `alerts_bp`.
- **6.5 Preparar para a apresentação parcial (29/05):**
 - Garantir que a API está rodando sem erros.
 - Ter exemplos de requisições prontos (curl/Postman) para demonstrar.
 - Alinhar com Frontend e Agente para que os dados fluam corretamente.

✓ Entregáveis da Semana

- Regra de Port Scan implementada.
- API organizada com Blueprints.
- Pronto para demonstrar na parcial.

♦ Semana 7 | 02 – 05 jun 🚀 MVP Deadline

🎯 **Meta da Semana:** Estabilidade, tratamento de erros e documentação final.

📋 Tarefas

- **7.1** Implementar validação de payload em todos os endpoints (usando `Flask-Marshmallow` ou validação manual).
- **7.2** Tratar edge cases:
 - Payload malformatado → retornar `400 Bad Request` com mensagem clara.
 - Banco de dados indisponível → retornar `503 Service Unavailable`.
- **7.3** Adicionar autenticação simples via **token estático** no header `X-API-Key` (apenas para evitar requisições acidentais; não é segurança forte, mas atende ao MVP).
- **7.4** Configurar **CORS** para permitir requisições do frontend (usando `Flask-CORS`).
- **7.5** Escrever `README.md` do backend com:
 - Como subir com Docker.
 - Descrição dos endpoints.
 - Exemplos de requisições.
- **7.6** Testar integração completa: Agente → Backend → Banco → Frontend.

✓ Entregáveis da Semana

- API estável, documentada e pronta para produção (MVP).

- Código limpo e organizado.
-

Período de Pitch (06 – 12 jun)

Tarefas do Backend para o Pitch

- **P.1** Garantir que a API está no ar durante a apresentação.
 - **P.2** Apoiar a demonstração de detecção de ataques (verificar se os alertas estão sendo gerados).
 - **P.3** Estar disponível para explicar a arquitetura e as regras de detecção implementadas.
-

Observações Técnicas

1. Docker Compose:

O serviço `backend` deve depender do serviço `db` (`depends_on: db`). Use `wait-for-it.sh` ou `healthcheck` para garantir que o banco esteja pronto antes de iniciar o Flask.

2. Persistência do Banco:

Certifiquem-se de que o volume do PostgreSQL está mapeado no `docker-compose.yml` para não perder dados.

3. Segurança (Token):

A autenticação por token estático é suficiente para o MVP. Em produção, usaríamos JWT ou OAuth2, mas não é necessário agora.

4. Performance:

Para evitar que o banco cresça infinitamente, coordenem com Beatriz a política de retenção de dados (ex.: deletar métricas com mais de 7 dias).